```
"""
Slayd bot (Telegram) — AI ishlatilmaydi.
Matn: Wikipedia, rasm: Wikimedia Commons. Chiqish: PPTX, PPT, PDF.

O'rnatish:
    pip install aiogram python-pptx pillow aiohttp
    + LibreOffice (soffice) — PPT va PDF uchun
Ishga tushirish:
    export BOT_TOKEN=...  ADMIN_ID=123456
    python slayd_bot.py
"""
import asyncio, io, json, os, re, subprocess, tempfile
import aiohttp
from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command, CommandStart
from aiogram.types import (CallbackQuery, FSInputFile, InlineKeyboardButton,
                           InlineKeyboardMarkup, Message)
from PIL import Image
from pptx import Presentation
from pptx.dml.color import RGBColor
from pptx.enum.shapes import MSO_SHAPE
from pptx.enum.text import MSO_ANCHOR
from pptx.util import Inches, Pt

TOKEN = os.getenv("BOT_TOKEN", "")
ADMIN = int(os.getenv("ADMIN_ID", "0"))
MIN_S, MAX_S = 2, 40

# ---- Tariflar: (max slayd, formatlar, narx) ----
PLANS = {
    "free":     (10, ["pptx"],               "Bepul"),
    "standard": (25, ["pptx", "pdf"],        "29 000 so'm"),
    "premium":  (40, ["pptx", "ppt", "pdf"], "59 000 so'm"),
}
DB = "users.json"

# ---- 12 shablon: nom, fon, sarlavha, matn, aksent, shrift ----
TPL = [
    ("Klassik",   "FFFFFF", "1F3864", "222222", "2E75B6", "Calibri"),
    ("Qorong'u",  "1E1E2E", "FFFFFF", "E0E0E0", "F5A623", "Calibri"),
    ("Yashil",    "F3F8F1", "1B5E20", "1F2A1F", "43A047", "Georgia"),
    ("Qizil",     "FFF5F5", "8E1B1B", "2B1F1F", "D32F2F", "Arial"),
    ("Binafsha",  "F6F2FA", "4A148C", "2A2233", "8E24AA", "Calibri"),
    ("Dengiz",    "EAF6FA", "01579B", "17252E", "00ACC1", "Verdana"),
    ("Qum",       "FBF6EC", "5D4037", "2E2622", "C58B3F", "Georgia"),
    ("Grafit",    "F2F2F2", "263238", "212121", "607D8B", "Arial"),
```

```
    ("Qora-oq",   "FFFFFF", "000000", "1A1A1A", "000000", "Arial"),
    ("Moviy tun", "0D1B2A", "E0E1DD", "D5D9E0", "778DA9", "Calibri"),
    ("Zumrad",    "ECF8F4", "004D40", "13261F", "00897B", "Verdana"),
    ("Olov",      "FFF8F0", "BF360C", "2D2118", "FB8C00", "Calibri"),
]

SKIP = {"manbalar", "havolalar", "izohlar", "adabiyotlar", "references",
        "external links", "see also", "notes", "sources", "bibliography"}
H = {"User-Agent": "SlideBot/1.0 (education)"}


# ===================== MA'LUMOT YIG'ISH =====================
async def get_text(ses, topic):
    for lang in ("uz", "en"):
        p = dict(action="query", format="json", prop="extracts", explaintext=1,
                 redirects=1, generator="search", gsrsearch=topic, gsrlimit=1)
        async with ses.get(f"https://{lang}.wikipedia.org/w/api.php", params=p) as r:
            pages = (await r.json()).get("query", {}).get("pages", {})
        for pg in pages.values():
            if len(pg.get("extract", "")) > 600:
                return pg["extract"], pg["title"]
    return "", topic


async def get_images(ses, topic, n):
    p = dict(action="query", format="json", generator="search", gsrnamespace=6,
             gsrsearch=topic, gsrlimit=50, prop="imageinfo",
             iiprop="url|mime|size", iiurlwidth=900)
    async with ses.get("https://commons.wikimedia.org/w/api.php", params=p) as r:
        pages = (await r.json()).get("query", {}).get("pages", {})
    urls = []
    for pg in sorted(pages.values(), key=lambda x: x.get("index", 0)):
        i = (pg.get("imageinfo") or [{}])[0]
        if i.get("mime") in ("image/jpeg", "image/png") and i.get("width", 0) >= 500:
            urls.append(i.get("thumburl") or i["url"])

    async def dl(u):
        try:
            async with ses.get(u) as r:
                return await r.read() if r.status == 200 else None
        except Exception:
            return None
    got = await asyncio.gather(*[dl(u) for u in urls[: n + 6]])
    return [g for g in got if g][:n]


def make_parts(text, need):
    """Matnni bo'limlarga bo'lib, har slaydga 3-4 ta gap beradi."""
    secs = re.split(r"\n==+\s*(.+?)\s*==+\s*\n", "\n" + text)
    pairs = [("Kirish", secs[0])] + list(zip(secs[1::2], secs[2::2]))
```

```
    out = []
    for title, body in pairs:
        if title.strip().lower() in SKIP:
            continue
        body = re.sub(r"\([^)]*\)|\[[^\]]*\]|\s+", " ", body).strip()
        sents = [s for s in re.split(r"(?<=[.!?])\s+", body) if 40 <= len(s) <= 260]
        chunks = [sents[i:i + 4] for i in range(0, len(sents), 4)]
        for k, c in enumerate(chunks):
            if len(c) >= 3:
                out.append((title if k == 0 else f"{title} ({k + 1})", " ".join(c)))
    return out[:need]


# ==================== SLAYD YASASH ====================
def rgb(h): return RGBColor.from_string(h)


def mix(a, b, t):
    A = [int(a[i:i + 2], 16) for i in (0, 2, 4)]
    B = [int(b[i:i + 2], 16) for i in (0, 2, 4)]
    return "".join(f"{int(x * (1 - t) + y * t):02X}" for x, y in zip(A, B))


def shape(s, kind, x, y, w, h, color):
    sh = s.shapes.add_shape(kind, Inches(x), Inches(y), Inches(w), Inches(h))
    sh.fill.solid(); sh.fill.fore_color.rgb = rgb(color); sh.line.fill.background()
    return sh


def text(s, t, x, y, w, h, size, color, font, bold=False, anchor=MSO_ANCHOR.TOP):
    tb = s.shapes.add_textbox(Inches(x), Inches(y), Inches(w), Inches(h))
    tf = tb.text_frame; tf.word_wrap = True; tf.vertical_anchor = anchor
    r = tf.paragraphs[0].add_run(); r.text = t
    r.font.size = Pt(size); r.font.bold = bold; r.font.name = font
    r.font.color.rgb = rgb(color)
    tf.paragraphs[0].line_spacing = 1.15


def picture(s, data, x, y, w, h):
    iw, ih = Image.open(io.BytesIO(data)).size
    k = min(w / iw, h / ih); pw, ph = iw * k, ih * k
    s.shapes.add_picture(io.BytesIO(data), Inches(x + (w - pw) / 2),
                         Inches(y + (h - ph) / 2), Inches(pw), Inches(ph))


def build(topic, parts, imgs, tpl, path):
    _, bg, tc, xc, ac, font = tpl
    soft = mix(ac, bg, 0.82)
    prs = Presentation(); prs.slide_width = Inches(13.333); prs.slide_height =
Inches(7.5)
```

```
    def base():
        s = prs.slides.add_slide(prs.slide_layouts[6])
        s.background.fill.solid(); s.background.fill.fore_color.rgb = rgb(bg)
        shape(s, MSO_SHAPE.OVAL, 11.0, -1.4, 3.6, 3.6, soft)       # suzuvchi
shakllar
        shape(s, MSO_SHAPE.OVAL, -0.9, 6.3, 2.2, 2.2, soft)
        return s

    # Titul slayd
    s = base()
    shape(s, MSO_SHAPE.RECTANGLE, 0.8, 2.3, 0.12, 2.2, ac)
    text(s, topic, 1.1, 2.1, 6.8 if imgs else 11, 2.6, 40, tc, font, True,
MSO_ANCHOR.MIDDLE)
    text(s, f"{len(parts) + 1} ta slayd", 1.1, 4.8, 6, 0.6, 18, ac, font)
    if imgs:
        picture(s, imgs[0], 8.2, 1.5, 4.5, 4.5)

    # Mazmun slaydlari
    for i, (title, body) in enumerate(parts):
        s = base()
        img = imgs[i + 1] if i + 1 < len(imgs) else None
        left = i % 2 == 0                          # rasm o'ng/chap almashadi
        tx = 0.8 if left or not img else 5.6
        tw = 7.0 if img else 11.7
        text(s, title, tx, 0.5, tw, 1.0, 30, tc, font, True, MSO_ANCHOR.MIDDLE)
        shape(s, MSO_SHAPE.RECTANGLE, tx + 0.05, 1.55, 1.4, 0.07, ac)
        text(s, body, tx, 1.9, tw, 4.8, 18, xc, font)
        if img:
            picture(s, img, 8.2 if left else 0.8, 1.7, 4.4, 4.6)
        text(s, str(i + 2), 12.3, 6.9, 0.7, 0.4, 12, ac, font)
    prs.save(path)


def convert(src, fmt, outdir):
    subprocess.run(["soffice", "--headless", "--convert-to", fmt, "--outdir", outdir,
src],
                   check=True, timeout=180, capture_output=True)
    return os.path.join(outdir, os.path.splitext(os.path.basename(src))[0] + "." +
fmt)


# ==================== BOT ====================
def load():
    return json.load(open(DB)) if os.path.exists(DB) else {}


def plan_of(uid): return load().get(str(uid), "free")
```

```
def set_plan(uid, plan):
    d = load(); d[str(uid)] = plan; json.dump(d, open(DB, "w"))


bot = Bot(TOKEN); dp = Dispatcher(); ST = {}


@dp.message(CommandStart())
async def start(m: Message):
    ST[m.from_user.id] = {"step": "topic"}
    await m.answer("Salom! Slayd mavzusini yozing.\n/tarif — tariflar")


@dp.message(Command("tarif"))
async def tarif(m: Message):
    cur = plan_of(m.from_user.id)
    rows = [f"{'✅ ' if k == cur else ''}{k.capitalize()} — {v[2]}\n"
            f"   {MIN_S}–{v[0]} slayd, {', '.join(f.upper() for f in v[1])}"
            for k, v in PLANS.items()]
    await m.answer("Tariflar:\n\n" + "\n\n".join(rows) +
                   "\n\nSotib olish uchun admin bilan bog'laning.")


@dp.message(Command("setplan"))
async def setplan(m: Message):                      # /setplan 123456 premium
    if m.from_user.id != ADMIN:
        return
    try:
        _, uid, plan = m.text.split()
        assert plan in PLANS
        set_plan(uid, plan); await m.answer("OK")
    except Exception:
        await m.answer("/setplan user_id free|standard|premium")


@dp.message(F.text & ~F.text.startswith("/"))
async def flow(m: Message):
    uid = m.from_user.id
    st = ST.setdefault(uid, {"step": "topic"})
    if st["step"] == "count":
        mx = PLANS[plan_of(uid)][0]
        if not m.text.strip().isdigit() or not MIN_S <= int(m.text) <= MAX_S:
            return await m.answer(f"Son yozing: {MIN_S}–{MAX_S}")
        if int(m.text) > mx:
            return await m.answer(f"Sizning tarifingizda maksimum {mx} slayd.
/tarif")
        st["count"] = int(m.text); st["step"] = "tpl"
        b = [InlineKeyboardButton(text=f"{i + 1}. {t[0]}", callback_data=f"t{i}")
             for i, t in enumerate(TPL)]
        kb = [b[i:i + 2] for i in range(0, 12, 2)]
```

```
        return await m.answer("Shablonni tanlang:",
reply_markup=InlineKeyboardMarkup(inline_keyboard=kb))
    st.update(topic=m.text.strip(), step="count")
    await m.answer(f"Necha slayd kerak? ({MIN_S}–{MAX_S})")


@dp.callback_query(F.data.startswith("t"))
async def make(c: CallbackQuery):
    uid = c.from_user.id; st = ST.get(uid)
    if not st or st.get("step") != "tpl":
        return await c.answer()
    st["step"] = "topic"; tpl = TPL[int(c.data[1:])]
    await c.message.edit_text("Tayyorlanmoqda, kuting...")
    try:
        async with aiohttp.ClientSession(headers=H) as ses:
            raw, title = await get_text(ses, st["topic"])
            parts = make_parts(raw, st["count"] - 1)
            if not parts:
                return await c.message.edit_text("Bu mavzu bo'yicha ma'lumot
topilmadi. Boshqa mavzu yozing.")
            imgs = await get_images(ses, st["topic"], len(parts) + 1)
        with tempfile.TemporaryDirectory() as d:
            safe = re.sub(r"\W+", "_", st["topic"])[:30] or "slayd"
            pptx = os.path.join(d, safe + ".pptx")
            build(title if len(title) < 60 else st["topic"], parts, imgs, tpl, pptx)
            for f in PLANS[plan_of(uid)][1]:
                p = pptx if f == "pptx" else await asyncio.to_thread(convert, pptx,
f, d)
                await c.message.answer_document(FSInputFile(p))
        await c.message.answer(f"Tayyor: {len(parts) + 1} ta slayd. Yangi mavzu
yozing.")
    except Exception as e:
        await c.message.answer(f"Xatolik: {e}")


if __name__ == "__main__":
    asyncio.run(dp.start_polling(bot))
```